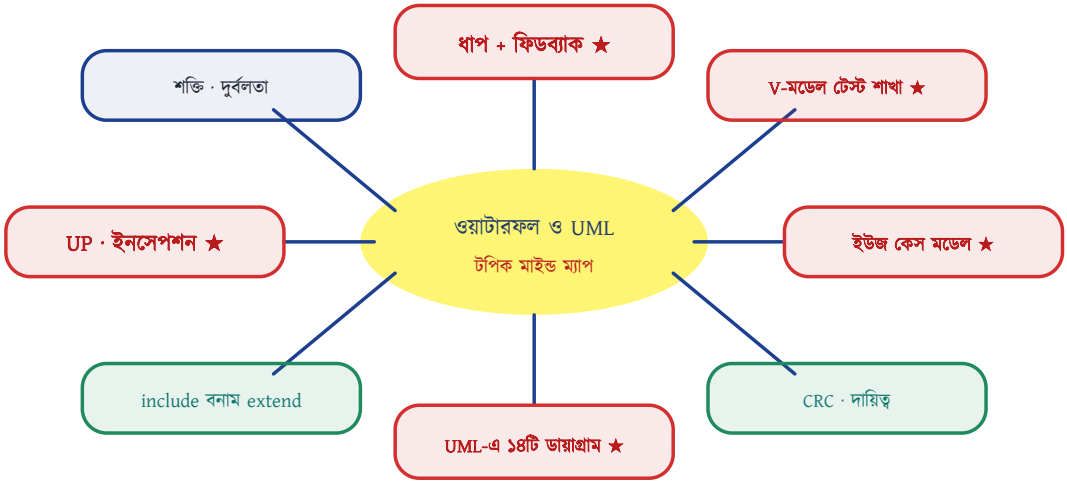
ওয়াটারফল, ইনসেপশন ও UML

ধাপ · V-মডেল · ইনসেপশন · ইউজ কেস · UML · OO ডিজাইন: ১১ পাতায় পরীক্ষার পূর্ণ প্রস্তুতি



↑ পুরো টপিক এক পাতায়: লাল = বেশি আসে, সবুজ = পদ্ধতি / প্রয়োগ

সিলেবাসের যে অংশ

"...Sequential **waterfall life cycles, Inception. Use case model ... UML diagrams ... Designing Objects with responsibilities.**"

হাই-ইল্ড তালিকা

① ওয়াটারফল: রৈখিক, কঠোর
② ফিডব্যাক আসে দেরিতে
③ V-মডেল = টেস্টের শাখা
④ ইনসেপশন ≈ ১০% প্রচেষ্টা
⑤ include বাধ্যতামূলক
⑥ ভরাট ◆ = কম্পোজিশন

সংকেত: **হলুদ** = মূল শব্দ · **লাল বৃত্ত** = মুখস্থ রাখার তথ্য · **বেগুনি বক্স** = মনে রাখার কৌশল · **লাল বক্স** = পরীক্ষার ফাঁদ · **সবুজ** = সমাধানকৃত উদাহরণ

সূচিপত্র

✓ ওয়াটারফল মডেল ও ধাপ ..... পৃ. ২
✓ শক্তি, দুর্বলতা, V-মডেল ..... পৃ. ৩
✓ ইউনিফাইড প্রসেস ও ইনসেপশন ..... পৃ. ৪
✓ ইনসেপশন বনাম প্রয়োজনীয়তা ধাপ ..... পৃ. ৫
✓ ইউজ কেস মডেল ..... পৃ. ৬
✓ include/extend, SSD, সিকোয়েন্স ..... পৃ. ৭
✓ UML: ১৪টি ডায়াগ্রাম ও সম্পর্ক ..... পৃ. ৮
✓ OO ডিজাইন ও CRC কার্ড ..... পৃ. ৯
✓ পরীক্ষার আগের রাতের পাতা ..... পৃ. ১০
✓ অনুশীলন MCQ + লিখিত ..... পৃ. ১১

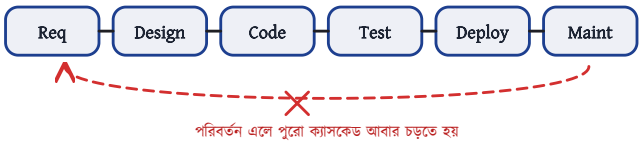
১ ওয়াটারফল (রৈখিক ক্রমিক) মডেল

সংজ্ঞা: উন্নয়ন একটি **ধাপের ধাপে (cascade)**: প্রতিটি ধাপ **সম্পন্ন, পর্যালোচিত ও অনুমোদিত** হওয়ার পরেই পরেরটি শুরু হয়, এবং এক ধাপের ডেলিভারেবলই পরেরটির একমাত্র ইনপুট। ধাপগুলো ওভারল্যাপ করে না, আর শেষ ধাপ আবার খোলা হয় না।

> n-এর আউটপুট = শুধু n+1-এর ইনপুট

#	ধাপ	প্রধান কাজ	ডেলিভারেবল
1	**প্রয়োজনীয়তা বিশ্লেষণ ও বিশেষীকরণ**	আহরণ, বিশ্লেষণ, SRS, সম্ভাব্যতা	SRS (ব্যবহারকারী + সিস্টেম)
2	**সিস্টেম ও সফটওয়্যার ডিজাইন**	আর্কিটেকচার, ডেটা, মডিউল, UI ডিজাইন	সিস্টেম ও সফটওয়্যার ডিজাইন ডকুমেন্ট
3	**বাস্তবায়ন (কোডিং)**	কোডিং, কোড রিভিউ, ইউনিট টেস্টিং	সোর্স কোড, বিল্ড
4	**ইন্টিগ্রেশন ও সিস্টেম টেস্টিং**	ইন্টিগ্রেশন, ফাংশনাল টেস্টিং, সংশোধন	টেস্ট প্ল্যান, রিপোর্ট, ত্রুটি লগ
5	**ডিপ্লয়মেন্ট**	ইনস্টল, মাইগ্রেশন, প্রশিক্ষণ	মুক্ত সিস্টেম, ম্যানুয়াল
6	**রক্ষণাবেক্ষণ (Maintenance)**	সংশোধন, অভিযোজন, উন্নতি	প্যাচ, হালনাগাদ ডকুমেন্ট

- **রৈখিক ও কঠোর** — একসময় একটি ধাপ সক্রিয়, কোনো ওভারল্যাপ নেই
- **ডকুমেন্টেশন-ভারী** — প্রতিটি ধাপ শেষ হয় পর্যালোচিত ডেলিভারেবলে
- **প্রয়োজনীয়তা ফ্রিজ** — পরের পরিবর্তন চেঞ্জ কন্ট্রোলের মধ্য দিয়ে যায়
- **যাচাই দেরিতে** — ফিডব্যাক থাকে **টেস্টিং ফেজে**



ছয়টি ধাপ, শুধু এক দিকে

মনে রাখার কৌশল: R-D-C-T-D-M: Requirements (প্রয়োজনীয়তা) → Design (ডিজাইন) → Code (কোড) → Test (টেস্ট) → Deploy (ডিপ্লয়) → Maintenance (রক্ষণাবেক্ষণ)। তুমি কেবল **নামার দিকে** যেতে পারো।

পরীক্ষার ফাঁদ: Royce-এর ১৯৭০ সালের কাগজে **আগের ধাপে ফিরে যাওয়া যাচাই-বিশিষ্ট তীর** আঁকা ছিল — ফিডব্যাকবিহীন ওয়াটারফল কেবল বইয়ের সরলীকরণ।

IT link: একটি সরকারি ই-টেন্ডার (টেন্ডার → SRS → নির্দিষ্ট মূল্যের চুক্তি → মাইলস্টোন) আদতন ওয়াটারফল: টাকা বাঁধার আগে ক্রেতাকে ফ্রিজ স্কোপ ডকুমেন্টের মালিক হতে হয়।

২ শক্তি, দুর্বলতা ও ফিডব্যাকের সমস্যা

সুবিধা:

- পরিকল্পনা ও **ব্যবস্থাপনা** করা সহজ
- **মাইলস্টোন ও ডেলিভারেবল** স্পষ্ট → অগ্রগতি মাপা যায়
- প্রতিটি ধাপের জন্য **বিশেষজ্ঞ** জনবল
- পুরো **ডকুমেন্টেশন** — হ্যান্ডওভার, অডিট, রক্ষণাবেক্ষণ
- **ছোট ও স্থির প্রয়োজনীয়তার** প্রকল্পে মানানসই (BPO, নির্দিষ্ট মূল্যের টেন্ডার)

অসুবিধা:

- ওভারল্যাপ নেই → **সময় দীর্ঘ**, কার্যকর সফটওয়্যার দেরিতে
- পরিবর্তন **ব্যয়বহুল**: ধাপে ফিরলে পরের সব ডকুমেন্ট বাতিল
- প্রয়োজনীয়তার ভুল কেবল **টেস্টিংয়ে** ধরা পড়ে
- ব্যবহারকারী শেষ পর্যন্ত **চলমান কিছু দেখে না**
- **বড়, নতুন বা দ্রুত বদলে যাওয়া** সিস্টেমে মানানসই নয়

ফিডব্যাকের সমস্যা: ওয়াটারফল ধরে নেয় প্রয়োজনীয়তা **কোড লেখার আগেই সম্পূর্ণ ও সঠিকভাবে বলা যায়**। কিন্তু ব্যবহারকারী সফটওয়্যার **চালিয়ে** দেখেই বিচার করতে পারে, তাই প্রথম সত্যিকার ফিডব্যাক আসে বাজেটের বেশিরভাগ খরচ হয়ে যাওয়ার পর — আর দেরির পরিবর্তন মানে ডিজাইন, কোড ও টেস্ট আবার করা।

> ত্রুটি খুঁজে বের করার খরচ বহুগুণ: প্রয়োজনীয়তা ১× · ডিজাইন ৫× · কোড ১০× · টেস্ট ১৫–৫০× · প্রোডাকশন ১০০–১০০০×



প্রতিটি উন্নয়ন লেভেলের সঙ্গে যে টেস্ট তা প্রমাণ করে

কোথায় এখনো মানানসই	কোথায় ব্যর্থ
ছোট প্রকল্প · স্থির প্রয়োজনীয়তা	অস্পষ্ট বা ওঠা-নামা প্রয়োজনীয়তা
প্রমাণিত প্রযুক্তি · নথিভুক্ত কাজ	বড়, বিতরণিত, দীর্ঘস্থায়ী সিস্টেম
নির্দিষ্ট মূল্যের মাইলস্টোন চুক্তি	উদ্ভাবনের ঝুঁকি · দ্রুত বাজার · ভারী UI কাজ

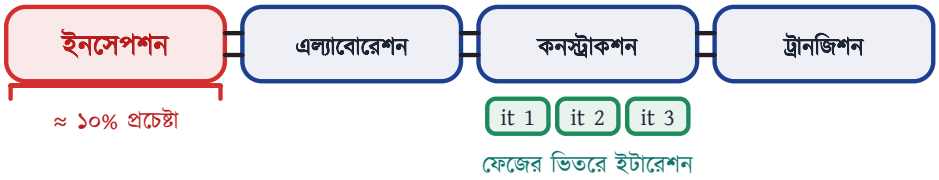
V-মডেল ✍️: একই ক্রমিক উন্নয়ন ধাপগুলো **বাম নামার শাখায়** রাখো আর মিলে যাওয়া **টেস্ট লেভেল ডান ওঠার শাখায়**। এটি ঠিক করে টেস্টিং **কোথায়** পরিকল্পনা করা হবে, ফিডব্যাক **কখন** আসবে তা নয়। **যাচাইকরণ (Verification)** = পণ্যটি কি *ঠিকভাবে* বানাচ্ছি; **বৈধতা (Validation)** = আমরা কি *ঠিক* পণ্যটি বানাচ্ছি। **টেস্টিং** খুঁজে দেয় ভুল আছে; **ডিবাগিং** সেগুলো সংশোধন করে।

পরীক্ষার ফাঁদ: V-মডেল মানে "প্রতি ইটারেশনে দেরিতে টেস্ট" নয় — এটি তখনও ক্রমিক; এবং **প্রয়োজনীয়তার প্রতি-পক্ষ হলো গ্রহণযোগ্যতা পরীক্ষা (acceptance test)**।

৩ ইউনিফাইড প্রসেস ও তার ইনসেপশন ফেজ

ইউনিফাইড প্রসেস (UP): একটি **ইউজ কেস-চালিত, আর্কিটেকচার-কেন্দ্রিক, পুনরাবৃত্তিমূলক ও ধাপে ধাপে বর্ধনশীল** প্রসেস **ফ্রেমওয়ার্ক**, Rational-এ Booch, Rumbaugh ও Jacobson কর্তৃক Cockburn-এর সহযোগিতায় তৈরি। প্রতিটি প্রকল্প একে **নিজের মতো সাজায় (tailors)**; Rational Unified Process (RUP) হলো Rational-এর প্যাকেজ করা সংস্করণ।

ফেজ	এতে যা হয়	মাইলস্টোন
ইনসেপশন (Inception)	ব্যবসায়িক কেস, ভিশন, প্রাথমিক স্কোপ, মোটামুটি অনুমান, প্রথম ঝুঁকি তালিকা	**ভিশন** (Vision) — "করা কি সার্থক?"
এল্যাবোরেশন (Elaboration)	অধিকাংশ প্রয়োজনীয়তা, ডোমেন মডেল, আর্কিটেকচারের প্রোটোটাইপ	**আর্কিটেকচার বেসলাইন**
কনস্ট্রাকশন (Construction)	বাকিটা তৈরি, ডিজাইন পরিশীলন, নিরবচ্ছিন্ন টেস্টিং	কার্যকর **ইনক্রিমেন্ট**
ট্রানজিশন (Transition)	ডিপ্লয়মেন্ট, মাইগ্রেশন, প্রশিক্ষণ, সাপোর্ট হ্যান্ডওভার	**মুক্ত পণ্য**



ফেজ = দীর্ঘমেয়াদি লক্ষ্য · ইটারেশন = ছোট চক্র

চারটি ফেজ, প্রতিটি ইটারেশনে পুনরাবৃত্তি; ফেজের মাইলস্টোন একটি প্রশ্নের উত্তর দেয়

ইনসেপশন: প্রথম ও সবচেয়ে ছোট ফেজ। এর একমাত্র প্রশ্ন **"এই প্রকল্প কি আদৌ শুরু করা উচিত?"** এটি ঠিক করে **ভিশন**, **ব্যবসায়িক কেস**, **প্রাথমিক স্কোপ**, একটি **মোটামুটি অনুমান** ও একটি **প্রাথমিক ঝুঁকি তালিকা**।

১. **স্কোপ ও আকার** ঠিক করা
২. **ব্যবসায়িক কেস** লিখে রাখা
৩. খরচ ও সময়ের মোটামুটি অনুমান
৪. **প্রাথমিক ঝুঁকি তালিকা** তৈরি
৫. **স্টেকহোল্ডার** চিহ্নিত করা
৬. **ভিশন** ও উচ্চ-স্তরের ইউজ কেস

ইনসেপশনের কার্যক্রম	যা তৈরি হয়
স্টেকহোল্ডার চিহ্নিতকরণ	ব্যবহারকারী, ক্লায়েন্ট, স্পনসর
প্রয়োজনীয়তা আহরণ	শুধু উচ্চ-স্তরের প্রয়োজন
সম্ভাব্যতা অধ্যয়ন	প্রযুক্তিগত, কার্যক্রম, আর্থিক
প্রাথমিক ঝুঁকি মূল্যায়ন	ঝুঁকি তালিকা, প্রভাব, প্রতিরোধ
উচ্চ-স্তরের ইউজ কেস	~১০–২০টি কেস, বিস্তারিত ছাড়া
প্রকল্প অনুমান	মোটামুটি আকার, খরচ, সময়সূচি

ইনসেপশনের আউটপুট: ভিশন ও ব্যবসায়িক কেস · প্রাথমিক ঝুঁকি তালিকা · প্রাথমিক উচ্চ-স্তরের ইউজ কেস মডেল · মোটামুটি আকার, খরচ ও সময়সূচি। **নির্দেশক হিসাব: প্রকল্পের প্রায় ১০% প্রচেষ্টা**; অনুমানটি **মোটামুটি**, সাধারণত **±৩০%**।

৪ ইনসেপশন বনাম ওয়াটারফলের প্রয়োজনীয়তা ধাপ

বিষয়	ওয়াটারফলের প্রয়োজনীয়তা	UP ইনসেপশন
কতটা স্কোপ	সম্পূর্ণ, বিস্তারিত, **ফ্রিজ**	শুধু উচ্চ-স্তরের, **ডিজাইন করেই অসম্পূর্ণ**
প্রধান আউটপুট	SRS	**ভিশন**, ব্যবসায়িক কেস, প্রাথমিক ইউজ কেস
সময়	ডিজাইনের আগে একটি দীর্ঘ ধাপ	ছোট প্রথম ফেজ; এল্যাবোরেশন তা এগিয়ে নেয়
নির্ভুলতা	হুবহু হওয়ার দাবি করে	মাত্রার ধারণা (order of magnitude), **±৩০%**
পরিবর্তন	চেঞ্জ রিকোয়েস্ট + প্রভাব বিশ্লেষণ	স্বাভাবিক — ইউজ কেস মডেল বাড়তেই থাকে
ঝুঁকি	স্পষ্টভাবে সম্বোধন হয় না	**প্রাথমিক ঝুঁকি তালিকা** একটি প্রথম-শ্রেণীর আউটপুট
টেস্টিং	পরে, V-মডেলের মাধ্যমে	নিরবচ্ছিন্ন, ঝুঁকিনির্ভর, প্রতি ইটারেশনে

সম্ভাব্যতা: পাঁচটি প্রশ্ন

- **প্রযুক্তিগত** — নাওয়া প্রযুক্তি ও দক্ষতায় কি এটি বানানো যাবে?

প্রাথমিক ঝুঁকি মূল্যায়ন

ঝুঁকি (Risk) হলো অনিশ্চিত ভবিষ্যৎ ঘটনা, যার একটি সম্ভাবনা ও একটি প্রভাব আছে; **এক্সপোজার = সম্ভাবনা × প্রভাব**। শৃঙ্খল **চিহ্নিতকরণ** → **মূল্যায়ন** → **প্রতিরোধ** → **পর্যবেক্ষণ**, আর

- **আর্থিক** — সুবিধা কি খরচের চেয়ে বেশি? (cost-benefit, payback, NPV)
- **কার্যক্রম** — মানুষ কি ব্যবহার করবে, প্রতিষ্ঠান কি চালাতে পারবে?
- **আইনি** — তথ্য সুরক্ষা ও ই-গভর্নমেন্ট সম্মতি
- **সময়সূচি** — চাওয়া সময়ে ডেলিভারেবল দেওয়া যাবে?

অধ্যয়ন শেষ হয় একটি **গো / নো-গো** সিদ্ধান্তে — দুর্বল প্রকল্প বাতিল করাও বৈধ ফলাফল।

প্রতিরোধ হয় এড়ানো, কমানো, হস্তান্তর বা গ্রহণ করে। ইনসেপশন কেবল **প্রথম** ঝুঁকি তালিকা দেয়; পূর্ণ ঝুঁকি বিশ্লেষণ ও COCOMO আছে প্রকল্প ব্যবস্থাপনার নোটে।

> **মনে রাখার কৌশল:** Inception = **"SCOOPE"** — **S**takeholders (স্টেকহোল্ডার), feasibility (সম্ভাব্যতা), **C**ost/effort estimate (খরচ/প্রচেষ্টার অনুমান), **O**utput = ভিশন ও ইউজ কেস, **P**roject go/no-go, **E**licit only high-level needs (শুধু উচ্চ-স্তরের প্রয়োজন আহরণ)।

> **পরীক্ষার ফাঁদ:** লেখো না "Inception প্রয়োজনীয়তা ধাপের বদলে দেয়" — এটি কেবল প্রয়োজনীয়তাকে *কাঠামো দেয়*; আসল বড় অংশ এল্যাবোরেশনে আসে। আর UP-কে "নতুন নামে ওয়াটারফল" বলো না: UP **পুনরাবৃত্তিমূলক**, প্রতিটি ফেজে অনেক ইটারেশন, আর ওয়াটারফল কঠোরভাবে ক্রমিক।

> **IT link:** বাস্তব প্রকল্পে ঠিক এটাই হয়: এক-দুই সপ্তাহের স্পাইক থেকে একটি ভিশন নোট, কয়েকটি ইউজ কেস ও ঝুঁকি তালিকা বের হয় — তারপর অর্থায়নের সিদ্ধান্ত নেওয়া হয়।

> **ইনসেপশনের ভিতরে মোটামুটি অনুমান ✍️:** কাজের আকার বের করো **ফাংশন পয়েন্ট (Function Point)**, **ইউজ কেস পয়েন্ট** (জটিলতা অনুযায়ী প্রতিটি ইউজ কেসের ওজন), **সাদৃশ্য (analogy)** বা **বিশেষজ্ঞের রায়** দিয়ে; তারপর প্রচেষ্টা × হার ≈ খরচ। ইনসেপশনে শুধু **মাত্রার ধারণা** দরকার — নিশ্চিত সংখ্যা আসে এল্যাবোরেশনের পর। COCOMO ও পূর্ণ ঝুঁকি ম্যাট্রিক্স আছে প্রকল্প ব্যবস্থাপনার নোটে।

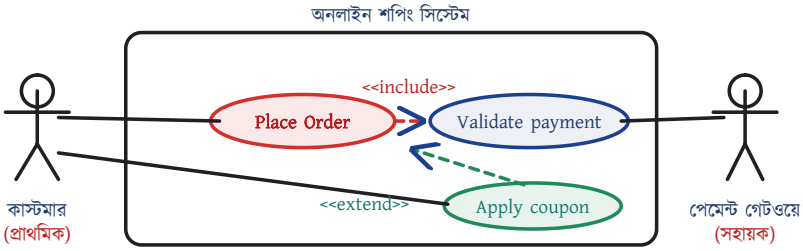
৫ প্রয়োজনীয়তা লেখার জন্য ইউজ কেস মডেল

> **সংজ্ঞা:** **ইউজ কেস (Use Case)** হলো এমন লেনদেনের ধারাবাহিকতা, যা কোনো নির্দিষ্ট অ্যাক্টরের সঙ্গে **অর্থবহ ফলাফল** দেয় (Ivar Jacobson)। এটি ব্যবহারকারীর ভাষায় **আচরণ** অনুযায়ী সারসংক্ষেপ করে — ডিজাইন, স্ক্রিন বা টেস্ট কেস নয়।

উপাদান	কী জানায়	"Place Order" উদাহরণ
অ্যাক্টর (Actor)	ব্যক্তি বা বাহ্যিক সিস্টেমের একটি **ভূমিকা**, সীমানার বাইরে, দাঁড়াই ছবির মতো আঁকা হয়	Customer; Payment Gateway
পূর্বশর্ত (Precondition)	আগে কী সত্য থাকতে হবে — একটি **অনুমান**, কখনও ট্রিগার নয়	নিবন্ধিত, লগ ইন করা, কার্ট খালি নয়
মূল প্রবাহ (Main flow)	লক্ষ্য পৌঁছানোর সাধারণ ক্রম, যা সবসময় কাজ করে	checkout → সারসংক্ষেপ → পেমেন্ট → নিশ্চিতকরণ
বিকল্প প্রবাহ	বৈচিত্র্য, ভুল ও ব্যতিক্রমের পথ	কার্ড বাতিল; স্টক বদলেছে; বাতিল
পরবর্তী শর্ত (Postcondition)	সফল হওয়ার পর কী সত্য থাকে — একটি **ফলাফল**	অর্ডার নিশ্চিত, স্টক সংরক্ষিত
ট্রিগার	ইউজ কেস শুরু করে এমন সীমানা-ঘটনা	কাস্টমার "Checkout" চাপে

- **প্রাথমিক অ্যাক্টর** — যার লক্ষ্য ইউজ কেস পূরণ করে; এটাই সিস্টেমকে চালায়
- **সহায়ক / গৌণ অ্যাক্টর** — ইউজ কেসের জন্য প্রয়োজনীয় সেবা দেয় (Payment Gateway); **তৃতীয়** অ্যাক্টর কেবল পরোক্ষ সুবিধা পায় (Courier)
- অ্যাক্টর একটি **ভূমিকা, ব্যক্তি নয়: একজন মানুষ কয়েকটি অ্যাক্টর হতে পারে**, একটি অ্যাক্টর কয়েকজন মানুষ হতে পারে।

সমাধানকৃত উদাহরণ: "Place Order" ✍️: মূল প্রবাহ: ① কাস্টমার Checkout নির্বাচন করে → ② সিস্টেম সারসংক্ষেপ দেখায় → ③ কাস্টমার কার্ডের তথ্য দেয় → ④ গেটওয়ে অনুমোদন করে → ⑤ সিস্টেম অর্ডার তৈরি করে, স্টক সংরক্ষণ করে ও নিশ্চিত করে → ⑥ অর্ডার আইডি দেখায়। **বিকল্প প্রবাহ:** কার্ড বাতিল → অন্য কার্ড; স্টক বদলেছে → পুনরায় হিসাব; বাতিল → কোনো অর্ডার নেই।



ইউজ কেস ডায়াগ্রাম: অ্যাক্টর সীমানার বাইরে, ডিম্ব ভেতরে

পরীক্ষার ফাঁদ: ইউজ কেস **নয়** কোনো স্ক্রিন, ক্লাস বা টেস্ট কেস। এর নাম দাও **ক্রিয়া + বিশেষ্য**, এবং **ব্যবহারকারীর লক্ষ্যের** লেভেলে।

মনে রাখার কৌশল: Precondition আগে, Main flow মাঝে, Postcondition পরে। অ্যাক্টর একটি **ভূমিকা**, বাক্সের বাইরে।

৬ ইউজ কেস সম্পর্ক, SSD ও সিকোয়েন্স ডায়াগ্রাম

সম্পর্ক	কখন ব্যবহার হয়	দিক ও চিহ্ন
<<include>>	**বাধ্যতামূলক** — বেস ইউজ কেস **সবসময়** অন্তর্ভুক্তটি সম্পাদন করে	বেস → অন্তর্ভুক্ত; ডটেড তীর, খোলা মাথা
<<extend>>	**ঐচ্ছিক** — শুধু কোনো শর্তে যোগ হয়; নিজে কখনও সম্পূর্ণ হয় না	সম্প্রসারক → সম্প্রসারিত
জেনারালাইজেশন	বিশেষায়িত সাধারণ প্রবাহ গ্রহণ করে ও নতুন ধাপ যোগ বা বদলায়	বিশেষায়িত → সাধারণ; ফাঁপা ত্রিভুজ

সিস্টেম সিকোয়েন্স ডায়াগ্রাম (SSD): পুরো সিস্টেমকে একটি কালো বাক্স হিসেবে আঁকে: কেবল সীমানা পার করা ঘটনাই দেখায়। নিয়ম (Cockburn): প্রতি ধাপে **একটি ঘটনা**, নাম **ক্রিয়া + বিশেষ্য** আকারে, **অভ্যন্তরীণ বার্তা নেই**, কোনো লুপ নেই।

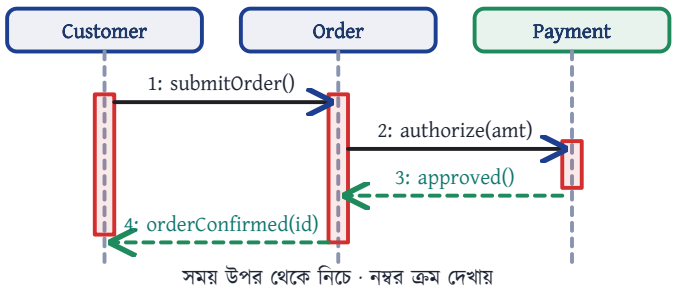
"Place Order"-এর SSD ✍️:

১. Customer → System: **createOrder(items)**
২. System → Customer: **paymentRequired(total)**
৩. Customer → System: **makePayment(cardDetails)**
৪. System → Customer: **orderConfirmed(orderId)**

গেটওয়ে কল এখানে বাক্সের **ভেতরে** — এই কারণেই SSD সিকোয়েন্স ডায়াগ্রাম নয়।

সিকোয়েন্স ডায়াগ্রাম পড়ার নিয়ম

- **সময় নিচের দিকে যায়**; প্রতিটি বার্তা **বাম → ডান** পড়ো, তারপর নামো।
- **লাইফলাইন (Lifeline)** = অংশ নেওয়া অবজেক্টগুলো।
- **নম্বরযুক্ত বার্তা** ক্রম দেখায়।
- **অ্যাক্টিভেশন বার** = নিয়ন্ত্রণের কেন্দ্র।
- **ভরাট তীর** = সমবর্তী · **খোলা তীর** = অসমবর্তী · **ডটেড** = উত্তর · [guard] = শর্ত।
- **বিন্দু** = বিদ্যমান · **খোলা বৃত্ত** = তৈরি · **X** = ধ্বংস।



SSD যা লুকিয়ে রাখে, সেটাই এখন খুলে দেখানো হলো

কমিউনিকেশন (= কোলাবোরেশন) ডায়াগ্রাম: **একই অন্তরচর্চা** যেটি **অবজেক্ট লিঙ্কের** গ্রাফ হিসেবে আবার আঁকা, লিঙ্কের উপর বার্তা নম্বরযুক্ত: এটি দেখায় **কে কার সঙ্গে যুক্ত** (গঠন), কখন কী ঘটে তা নয় (সময়)।

পরীক্ষার ফাঁদ: "Withdraw Cash **সবসময়** Authenticate User করে" → **<<include>>**। "শুধু **সদস্য** হলে ছাড়" → **<<extend>>**। তীর একই — **দিক ও শর্ত** এদের আলাদা করে।

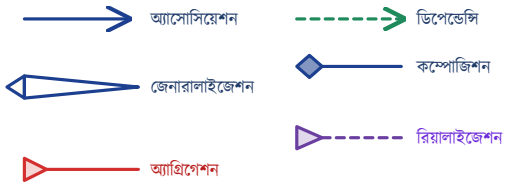
৭ UML: কোন ডায়াগ্রাম কী কাজে

UML: সফটওয়্যার নির্ধারণ, কল্পনা ও নথিভুক্তকরণের জন্য OMG-এর মানক ভিজ্যুয়াল ভাষা, ১৯৮০-এর শেষ দিকে Rational-এ তৈরি (Booch, Rumbaugh, Jacobson), ১৯৯৭ থেকে OMG-এর মান। UML 2-তে **১৪টি ডায়াগ্রাম: স্ট্রাকচারাল** (কী আছে) ও **বিহেভিয়ারাল** (কী ঘটে)।

স্ট্রাকচারাল (৭)	কী কাজে
ক্লাস (Class)	স্থির গঠন: ক্লাস, অ্যাট্রিবিউট, অপারেশন
অবজেক্ট (Object)	*ইনস্ট্যান্স* ও তাদের লিঙ্কের এক মুহূর্তের ছবি
কম্পোজিট স্ট্রাকচার	কোনো ক্লাসিফায়ার কীভাবে অংশ দিয়ে তৈরি
কম্পোনেন্ট (Component)	মডিউল, ইন্টারফেস, নির্ভরশীলতা
ডিপ্লয়মেন্ট	হার্ডওয়্যার নোড ও লিঙ্কের উপর আর্টিফ্যাক্ট
প্যাকেজ (Package)	নেমস্পেস / ফোল্ডারে ভাগ করা
প্রোফাইল (Profile)	নির্দিষ্ট ডোমেইনের জন্য UML-এর স্টেরিওটাইপ সাজানো

বিহেভিয়ারাল (৭)	কী কাজে
ইউজ কেস	অ্যাক্টর ও তাদের চাওয়া লক্ষ্য
অ্যাক্টিভিটি	ওয়ার্কফ্লো: প্রবাহ, সিদ্ধান্ত, সুইমলেন
স্টেট মেশিন	একটি অবজেক্টের জীবন: অবস্থা, ঘটনা
সিকোয়েন্স	সময়ের সঙ্গে অন্তরচর্চা: লাইফলাইন + বার্তা
কমিউনিকেশন	একই অন্তরচর্চা অবজেক্ট গ্রাফ হিসেবে
ইন্টারঅ্যাকশন ওভারভিউ	সব ইন্টারঅ্যাকশন ডায়াগ্রাম কীভাবে জোড়া লাগে
টাইমিং	সময়ের সঙ্গে অবজেক্টের মান বদলানো

সম্পর্ক	অর্থ → চিহ্ন
অ্যাসোসিয়েশন	"a জানে b"; ভূমিকা, নেভিগেবিলিটি, মাল্টিপ্লিসিটি → **সরু রেখা**
অ্যাগ্রিগেশন	দুর্বল whole-part, অংশ একা বাঁচে → **ফাঁপা হীরকফলা**
কম্পোজিশন	শক্তিশালী whole-part, এক মালিক → **ভরাট হীরকফলা**
ডিপেন্ডেন্সি	সবচেয়ে দুর্বল, অস্থায়ী "ব্যবহার করে" → **ডটেড + খোলা মাথা**
জেনারালাইজেশন	"is-a": সাবক্লাস → সুপারক্লাস → সরু রেখা, **ফাঁপা** △
রিয়ালাইজেশন	চুক্তি "বাস্তবায়ন" → ডটেড রেখা, **ফাঁপা** △



ফাঁপা △ জেনারালাইজ · ভরাট ◆ কম্পোজ · ◇ অ্যাগ্রিগ

ছয়টি চিহ্নই আঁকতে হবে

পরীক্ষার ফাঁদ: ভরাট হীরকফলা = **কম্পোজিশন**, ফাঁপা হীরকফলা = **অ্যাগ্রিগেশন**; ফাঁপা **ত্রিভুজ** জেনারালাইজেশন, হীরকফলা নয়। "কোলাবোরেশন ডায়াগ্রাম" হলো **কমিউনিকেশন ডায়াগ্রামের** পুরোনো UML 1.x নাম।

৮ অবজেক্ট ডিজাইন: নীতি, প্রক্রিয়া ও CRC

OO নীতি	ঠিক কী বোঝায়
এনক্যাপসুলেশন (information hiding)	ডেটা ও তার অপারেশন এক শ্রেণিতে বাঁধা; অবস্থা ব্যক্তিগত, শুধু public ইন্টারফেস দিয়েই বদলায়
অ্যাবস্ট্রাকশন	অবজেক্ট **কী** করে তা বাইরে দেখায়, **কীভাবে** করে তা নয়
ইনহেরিটেন্স	সাবক্লাস অবস্থা ও আচরণ উত্তরাধিকার ও বিশেষায়িত করে ("is-a")
পলিমরফিজম	**একই বার্তা**, ভিন্ন আচরণ, সিদ্ধান্ত হয় **রান টাইমে**

OO ডিজাইন প্রক্রিয়া

১. একটি **ডিজাইন মডেল** তৈরি বা পুনর্ব্যবহার করো
২. প্রতিটি উপাদানের **ডিজাইন ক্লাস** খুঁজে বের করো
৩. প্রতিটি ক্লাসকে তার **দায়িত্ব** দাও (CRC কার্ড)
৪. প্রতিটি ক্লাসের **সহযোগী শ্রেণি** ঠিক করো
৫. সহযোগিতা, অ্যাসোসিয়েশন, জেনারালাইজেশন ডিজাইন করো
৬. ক্লাস পরিশীলন করো; সব আচরণ ধরা পড়া না পর্যন্ত পুনরাবৃত্তি

দায়িত্ব (Responsibility) = একটি ক্লাস যা **জানে** (অ্যাট্রিবিউট, লিঙ্ককৃত অবজেক্ট) বা যা **করে** (অপারেশন); **সহযোগী (Collaborator)** হলো সেই অন্য ক্লাস যা তা পূরণে সাহায্য করে।

ক্লাস	জানে (অবস্থা)	করে (অপারেশন)
Customer	name, email, cart	addToCart(), checkout()
Cart	products, quantities	calculateTotal(), createOrder()
Order	orderId, lines	confirm(), generateInvoice()
Payment	method, amount	processTransaction()

CART	দায়িত্ব	সহযোগী শ্রেণি
	– পণ্যসমূহ জানে – মোট জানে – addItem(sku) – calculateTotal() – createOrder() – clear()	Customer Product Order Payment

একটি CRC কার্ড (Beck ও Cunningham, ১৯৮৬)

কনসেপ্ট বনাম ডিজাইন ক্লাস: **ডোমেন (কনসেপ্ট) ক্লাস** শুধু তথ্য — **গণনার অপারেশন নেই** — নাম একবচনের সাধারণ বিশেষ্যে রাখা হয়। **ডিজাইন ক্লাস** হলো ডিজাইন মডেলের যেকোনো ক্লাস — ইমপ্লিমেন্টেশন-লেভেলের, অ্যাট্রিবিউট ও অপারেশনের পূর্ণ বিবরণসহ; এটি বিমূর্ত (abstract) হতেই হবে না।

পুনর্ব্যবহার ও রিফ্যাক্টরিং: Reuse = প্যাটার্ন, ইনহেরিটেন্স, কম্পোজিশন। Refactoring = কোডের **আচরণ-রক্ষা করা** পুনর্গঠন, ছোট ছোট ধাপে, কখনও নতুন ফিচারের সঙ্গে মেশানো নয়।

অবজেক্টের শ্রেণি: Entity = ইউজ কেসের ডেটা (Order) · Boundary = যা ব্যবহারকারী স্পর্শ করে (CheckoutScreen) · Control = সিকোয়েন্সিংয়ের যুক্তি (PlaceOrderController)।

ডোমেন মডেল (Domain Model) ✍️: ডোমেন মডেল হলো বাস্তব জগতের ধারণাগুলোর (কনসেপ্ট ক্লাস) ও তাদের সম্পর্কের **ধারণাগত** উপস্থাপন — কোনো কোড বা পদ্ধতি ছাড়াই। অনলাইন শপিংয়ে Customer **1** → ***** Order, প্রতিটি Order-এর একাধিক OrderLine, আর OrderLine ***** → **1** Product। এই **মাল্টিপ্লিসিটি** (1, *, 0..1, 1..*) লিখতে বলা হয় এবং পরীক্ষায় প্রায়ই চাওয়া হয়।

পরীক্ষার আগের রাতের পাতা

ফ্যাক্ট শিট:

ধাপ: Req → Design → Code → Test → Deploy → Maint

Royce, ১৯৭০ · রৈখিক, কঠোর, ওভারল্যাপ নেই

যাচাইকরণ = ঠিকভাবে পণ্য বানানো

বৈধতা = ঠিক পণ্য বানানো

V-মডেল: প্রয়োজনীয়তা ↔ গ্রহণযোগ্যতা · আর্কিটেকচার ↔ ইন্টিগ্রেশন

বিস্তারিত ডিজাইন ↔ ইউনিট টেস্ট

টেস্টিং ভুল খোঁজে · ডিবাগিং ভুল সারায়

UP = ইউজ কেস চালিত, আর্কিটেকচার কেন্দ্রিক

UP: ইনসেপশন · এল্যাবোরেশন · কনস্ট্রাকশন · ট্রানজিশন

প্রতিটি ফেজে অনেক ইটারেশন

ইনসেপশন = ভিশন, ব্যবসায়িক কেস, স্কোপ, অনুমান, ঝুঁকি

ইনসেপশন ≈ ১০% প্রচেষ্টা · অনুমান ±৩০%

<<include>> = সবসময় (বেস → অন্তর্ভুক্ত)

<<extend>> = শুধু শর্তে (সম্প্রসারক → সম্প্রসারিত)

SSD = কালো বাক্স, ক্রিয়া+বিশেষ্য, অভ্যন্তরীণ নেই

সিকোয়েন্স: সময় ↓, লাইফলাইন, অ্যাক্টিভেশন

অ্যাগ্রিগেশন = ফাঁপা ◆ · কম্পোজিশন = ভরাট ◆

ডিপেন্ডেন্সি = ডটেড + খোলা তীর

জেনারালাইজেশন / রিয়ালাইজেশন = ফাঁপা △

UML 2 = ১৪টি ডায়াগ্রাম = ৭ + ৭

কোলাবোরেশন = কমিউনিকেশন ডায়াগ্রাম

CRC: Beck ও Cunningham, ১৯৮৬

দায়িত্ব = জানা + করা

রিফ্যাক্টরিং = আচরণ-রক্ষা করা

কনসেপ্ট ক্লাস = শুধু তথ্য

অবজেক্ট: এন্টিটি · বাউন্ডারি · কন্ট্রোল

MCQ-র জন্য সেরা ১৪টি এক-লাইনের তথ্য

1. ওয়াটারফল প্রস্তাব করেন W. W. Royce, ১৯৭০
2. ওয়াটারফল = রৈখিক-ক্রমিক, ধাপ ওভারল্যাপ করে না
3. ইনসেপশন UP-এর প্রথম ফেজ
4. ইনসেপশনের প্রশ্ন "শুরু করা কি সার্থক?"
8. কোলাবোরেশন ডায়াগ্রামের নাম এখন কমিউনিকেশন
9. ভরাট হীরকফলা = কম্পোজিশন
10. ফাঁপা হীরকফলা = অ্যাগ্রিগেশন
11. ফাঁপা ত্রিভুজ = জেনারালাইজেশন

5. ইনসেপশনের আউটপুট ভিশন, SRS নয়
6. অ্যাক্টর সিস্টেম সীমানার বাইরে
7. সিকোয়েন্স ডায়াগ্রাম = লাইফলাইন + বার্তা, সময়ের ক্রমে
12. ডটেড তীর, খোলা মাথা = ডিপেন্ডেন্সি
13. এল্যাবোরেশন আর্কিটেকচার বেসলাইন তৈরি করে
14. CRC কার্ড বানিয়েছেন Beck ও Cunningham

সেরা ফাঁদগুলো: <<include>> বনাম <<extend>>: "সবসময়" বনাম "শুধু শর্তে" · কম্পোজিশন বনাম অ্যাগ্রিগেশন: ভরাট বনাম ফাঁপা হীরকফলা · যাচাইকরণ বনাম বৈধতা: পণ্য বনাম প্রক্রিয়া · SSD বনাম সিকোয়েন্স: কালো বাক্স বনাম খোলা · "ওয়াটারফলে কোনো ফিডব্যাকই নেই" ভুল · ইনসেপশন SRS লেখে না · ইউজ কেস কোনো স্ক্রিন নয়।

সব কৌশল একসঙ্গে: R-D-C-T-D-M সিঁড়ি · "শুধু নামার দিকে" · V = বামে উন্নয়ন, ডানে টেস্ট · ইনসেপশন = SCOOPE · পূর্বশর্ত আগে, মূল প্রবাহ মাঝে, পরবর্তী শর্ত পরে · "সবসময়" = include, "শুধু শর্তে" = extend · সরু রেখা = জানে, ডটেড = কেবল ব্যবহার · ফাঁপা হীরকফলা দুর্বল, ভরাট শক্তিশালী · কনসেপ্ট ক্লাস জমায়, ডিজাইন ক্লাস আচরণ করে।



include ও extend — একই তীর, উল্টো কাজ

✓ অনুশীলন MCQ (পরীক্ষার ধাঁচে, ১০ নম্বর)

১. কোন ডায়াগ্রামে লাইফলাইন ও বার্তা ব্যবহার হয়?
ক) ক্লাস ডায়াগ্রাম খ) সিকোয়েন্স ডায়াগ্রাম
গ) ডিপ্লয়মেন্ট ঘ) ইউজ কেস

২. সিস্টেমের সঙ্গে যোগাযোগকারী ব্যক্তি বা বাহ্যিক সিস্টেমকে কী বলা হয়?
ক) ক্লাস খ) নোড
গ) অ্যাক্টর ঘ) কম্পোনেন্ট

৩. UP-এর কোন ফেজে ব্যবসায়িক কেস ঠিক হয়?
ক) ইনসেপশন খ) এল্যাবোরেশন
গ) কনস্ট্রাকশন ঘ) ট্রানজিশন

৪. একটি বেস ইউজ কেস **সবসময়** অন্য আরেকটি করে:
ক) <<extend>> খ) জেনারালাইজেশন
গ) অ্যাসোসিয়েশন ঘ) <<include>>

৫. কম্পোজিশন কোন চিহ্নে আঁকা হয়?
ক) ফাঁপা হীরকফলা খ) ভরাট হীরকফলা
গ) ফাঁপা ত্রিভুজ ঘ) ডটেড তীর

৬. অ্যাগ্রিগেশন কোন চিহ্নে আঁকা হয়?
ক) ফাঁপা হীরকফলা খ) ভরাট হীরকফলা
গ) ফাঁপা ত্রিভুজ ঘ) বিন্দুরেখা

৭. "নির্ভর করে" সম্পর্কটি কীভাবে আঁকা হয়?
ক) সরু তীর খ) ভরাট হীরকফলা
গ) ডটেড তীর ঘ) সরু ত্রিভুজ

৮. কুপন কোড থাকলেই কেবল চলে:
ক) <<include>> খ) <<extend>>
গ) জেনারালাইজেশন ঘ) রিয়ালাইজেশন

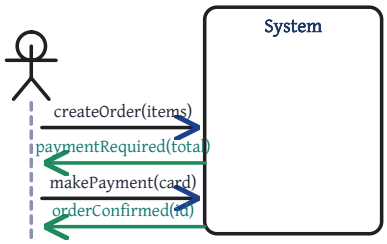
৯. "SavingsAccount is-a Account" কোন চিহ্নে দেখানো হয়?
ক) ভরাট হীরকফলা খ) ফাঁপা হীরকফলা
গ) ফাঁপা ত্রিভুজ ঘ) ডটেড তীর

১০. ইনসেপশনে প্রায় কত অংশ প্রচেষ্টা ব্যবহৃত হয়?
ক) 25% খ) 50%
গ) 10% ঘ) 75%

উত্তরমালা → ১-খ · ২-গ · ৩-ক · ৪-ঘ · ৫-খ · ৬-ক · ৭-গ · ৮-খ · ৯-গ · ১০-গ

লিখিত প্রশ্ন ১: ওয়াটারফল মডেল ও এর ধাপগুলো বর্ণনা করো। দুটি সুবিধা ও দুটি অসুবিধা লেখো। ৮ নম্বর

১. **সংজ্ঞা** — রৈখিক-ক্রমিক: পরের ধাপ শুরু হয় আগেরটি শেষ, পর্যালোচিত ও অনুমোদিত হওয়ার পর ২
২. **ধাপ** — প্রয়োজনীয়তা (SRS) → ডিজাইন → ইউনিট টেস্টিংসহ বাস্তবায়ন → ইন্টিগ্রেশন ও সিস্টেম টেস্টিং → ডিপ্লয়মেন্ট → রক্ষণাবেক্ষণ ২
৩. **সুবিধা** — সহজ পরিকল্পনা ও স্পষ্ট মাইলস্টোন; ভালো ডকুমেন্টেশন ২
৪. **অসুবিধা** — পরিবর্তন ব্যয়বহুল; কার্যকর সফটওয়্যার দেরিতে ২



SSD = একটি কালো বাক্স

লিখিত প্রশ্ন ২: অনলাইন শপিংয়ের ইউজ কেস ডায়াগ্রাম আঁকো — একটি প্রাথমিক অ্যাক্টর, একটি সহায়ক অ্যাক্টর, একটি <<include>> ও একটি <<extend>> সহ, এবং প্রতিটি তীরের দিক লেখো। ৭ নম্বর **উত্তর: কাস্টমার** বাঁয়ে (প্রাথমিক), **পেমেন্ট গেটওয়ে** ডানে (সহায়ক); <<include>> *Place Order → Validate Payment*; <<extend>> *Apply Coupon → Place Order*।

পরীক্ষার আগে যাচাই ✓

- ✓ ওয়াটারফলের ছয়টি ধাপ মুখস্থ করো
- ✓ দুটি সুবিধা ও দুটি অসুবিধা বলো
- ✓ UP-এর চারটি ফেজ ও ইনসেপশনের আউটপুট নামাও
- ✓ শর্ত দিয়ে include আর extend আলাদা করো
- ✓ UML-এর ছয়টি চিহ্নই আঁকতে পারো

পাশের লেসন: SDLC-এর ধাপ SE Concepts-এ; অনুমান ও COCOMO **প্রকল্প ব্যবস্থাপনা**-তে; দায়িত্বের প্যাটার্ন GRASP ও DCD-তে।